

Lab15-1 AI Assisted Coding

2303A51804

B-28

Task1:

Use AI to build a RESTful API for managing student records.

Prompt:

generate a python code to build a RESTful API for managing student records

The API should support the following operations:

GET/students - Retrieve a list of all students

POST/students - Create a new student record

PUT/students/{id} - Update an existing student record

DELETE/students/{id} - Delete a student record

use an in-memory data structure (list or dictionary) to store records and ensure API responses are in JSON format. Include error handling for cases such as trying to update or delete a non-existent student record.

CODE:

```
lab_15_1.py > C:\Drive\Documents\AI Assisted coding\lab11-1.py
1 #generate a python code to build a RESTful API for managing student records
2 # The API should support the following operations:
3 #GET/students - Retrieve a list of all students
4 #POST/students - Create a new student record
5 #PUT/students/{id} - Update an existing student record
6 #DELETE/students/{id} - Delete a student record
7 #use an in-memory data structure (list or dictionary) to store records and ensure API responses are in JSON format. Include error handling fo
8 from flask import Flask, jsonify, request
9 app = Flask(__name__)
10 students = {}
11 @app.route('/')
12 def home():
13     return jsonify({'message': 'Welcome to the Student Records API!'})
14 @app.route('/students', methods=['GET'])
15 def get_students():
16     return jsonify(list(students.values()))
17 @app.route('/students', methods=['POST'])
18 def create_student():
19     data = request.get_json()
20     student_id = data.get('id')
21     if student_id in students:
22         return jsonify({'error': 'Student already exists'}), 400
23     students[student_id] = data
24     return jsonify(data), 201
25 @app.route('/students/<int:student_id>', methods=['PUT'])
26 def update_student(student_id):
27     if student_id not in students:
28         return jsonify({'error': 'Student not found'}), 404
29
30
31
32
33
34
35
36
37
38
39
40
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

DOCTERMAN CONSOLE: TERMINAL

```
15 def get_students():
16     return jsonify(list(students.values()))
17 @app.route('/students', methods=['POST'])
18 def create_student():
19     data = request.get_json()
20     student_id = data.get('id')
21     if student_id in students:
22         return jsonify({'error': 'Student already exists'}), 400
23     students[student_id] = data
24     return jsonify(data), 201
25 @app.route('/students/<int:student_id>', methods=['PUT'])
26 def update_student(student_id):
27     if student_id not in students:
28         return jsonify({'error': 'Student not found'}), 404
29     data = request.get_json()
30     students[student_id] = data
31     return jsonify(data)
32 @app.route('/students/<int:student_id>', methods=['DELETE'])
33 def delete_student(student_id):
34     if student_id not in students:
35         return jsonify({'error': 'Student not found'}), 404
36     del students[student_id]
37     return jsonify({'message': 'Student deleted'})
38 if __name__ == '__main__':
39     app.run(debug=True)
40
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

OUTPUT:

Post:

http://127.0.0.1:5000/students

SaveNo Env

POST

http://127.0.0.1:5000/students

Params

Authorization

Headers (9)

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

1

2

3

4

5

{

"id": 1,

"name": "Pavani",

"age": 20

}

Body

Cookies

Headers (5)

Test Results

Status: 201 Created Time: 35 ms Size: 21

Pretty

Raw

Preview

JSON

1

2

3

4

5

{

"age": 20,

"id": 1,

"name": "Pavani"

}

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

POSTMAN CONSOLE:

TERMINAL

> POST http://127.0.0.1:5000/students

> POST http://127.0.0.1:5000/students

* Debugger PIN: 124-011-949

* Detected change in 'c:\\Users\\Pavani\\OneDrive\\Documents\\AI Assisted coding\\lab_15_1.py', reloading

127.0.0.1 - - [23/Mar/2026 10:17:45] "GET /students HTTP/1.1" 200 -

127.0.0.1 - - [23/Mar/2026 10:18:11] "GET /students HTTP/1.1" 200 -

GET

http://127.0.0.1:5000/students

GET http://127.0.0.1:5000/students

Params Authorization Headers (9) **Body** Scripts Settings

Body Cookies Headers (5) Test Results

Status: 200 OK Time: 6 ms Size: 406 B

Pretty Raw Preview JSON

```
1 [
2   {
3     "age": 21,
4     "id": 1,
5     "name": "Pavani"
6   },
7   {
8     "age": 20,
9     "id": 2,
10    "name": "Manisha"
11  },
12  {
13    "age": 21,
14    "id": 3,
15    "name": "Ashwitha"
16  },
17  ]
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

PUT

http://127.0.0.1:5000/students/1

PUT http://127.0.0.1:5000/students/1

Params Authorization Headers (9) **Body** Scripts Settings

Code Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

Beautify

```
1 {
2   "name": "Pallavi"
3 }
4
```

Body Cookies Headers (5) Test Results

Status: 200 OK Time: 8 ms Size: 190 B

Pretty Raw Preview JSON

```
1 {
2   "name": "Pallavi"
3 }
4
```

DELETE

HTTP <http://127.0.0.1:5000/students/2>

DELETE [▼](#) | <http://127.0.0.1:5000/students/2>

Params Authorization Headers (7) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL [JSON](#) [▼](#)

1

← ↻ 127.0.0.1:5000/students Summa

Pretty-print ☐

```
[
  {
    "age": 21,
    "id": 3,
    "name": "Ashwitha"
  },
  {
    "age": 15,
    "id": 4,
    "name": "Sanvika"
  }
]
```

EXPLANATION:

- Built a RESTful API using Flask to perform CRUD operations on student records.
- Used an in-memory data structure (dictionary/list) to store and manage student data.
- Implemented JSON responses with proper error handling for invalid requests and missing records.

Task 2 – Library Book Management API

Prompt:

develop a python code to develop a RESTful API to handle library booksThe API should support the following operations:

GET /books → Retrieve all books

POST /books → Add a new book

GET /books/{id} → Get details of a specific book

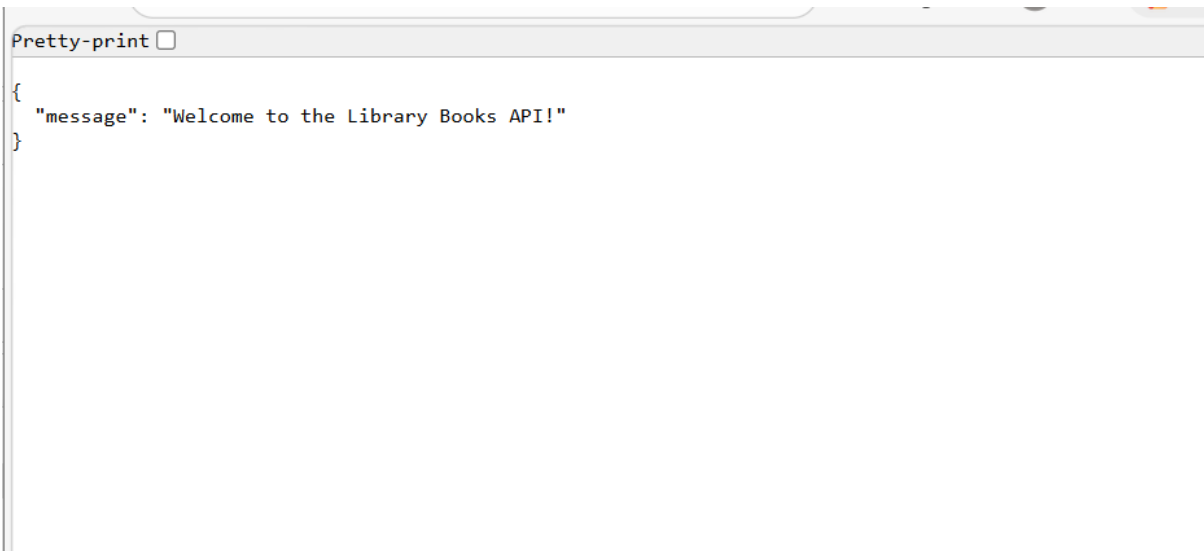
PATCH /books/{id} → Update partial book details (e.g.,availability)

DELETE /books/{id} → Remove a book and also implement error handling for invalid requests

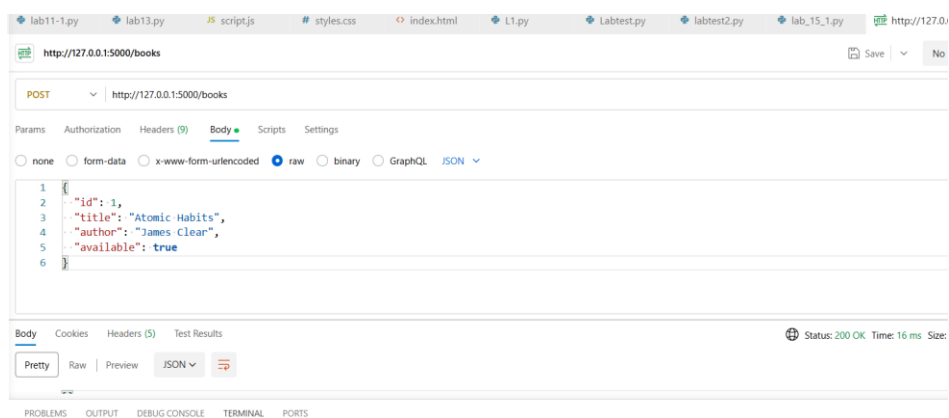
CODE:

```
lab_15_1.py > ...
42 #develop a python code to develop a RESTful API to handle library books
43 # The API should support the following operations:
44 #GET /books → Retrieve all books
45 #POST /books → Add a new book
46 #GET /books/{id} → Get details of a specific book
47 #PATCH /books/{id} → Update partial book details (e.g.,availability)
48 # DELETE /books/{id} → Remove a book
49 #also implement error handling for invalid requests
50 from flask import Flask, jsonify, request
51 app = Flask(__name__)
52 books = {}
53
54 @app.route('/', methods=['GET'])
55 def home():
56     return jsonify({"message": "Welcome to the Library Books API!"})
57 @app.route('/books', methods=['GET'])
58 def get_books():
59     return jsonify(list(books.values()))
60 @app.route('/books', methods=['POST'])
61 def add_book():
62     data = request.get_json()
63     book_id = data.get('id')
64     if book_id in books:
65         return jsonify({'error': 'Book already exists'}), 400
66     books[book_id] = data
67     return jsonify(data), 201
68 @app.route('/books/<int:book_id>', methods=['GET'])
69 def get_book(book_id):
70     if book_id not in books:
71         return jsonify({'error': 'Book not found'}), 404
72     return jsonify(books[book_id])
73 @app.route('/books/<int:book_id>', methods=['PATCH'])
74 def update_book(book_id):
75     if book_id not in books:
76         return jsonify({'error': 'Book not found'}), 404
77     data = request.get_json()
78     books[book_id].update(data)
79     return jsonify(books[book_id])
80 @app.route('/books/<int:book_id>', methods=['DELETE'])
81 def delete_book(book_id):
82     if book_id not in books:
83         return jsonify({'error': 'Book not found'}), 404
84     del books[book_id]
85     return jsonify({'message': 'Book deleted'})
86 if __name__ == '__main__':
87     app.run(debug=True,host='0.0.0.0', port=5000)
88
```

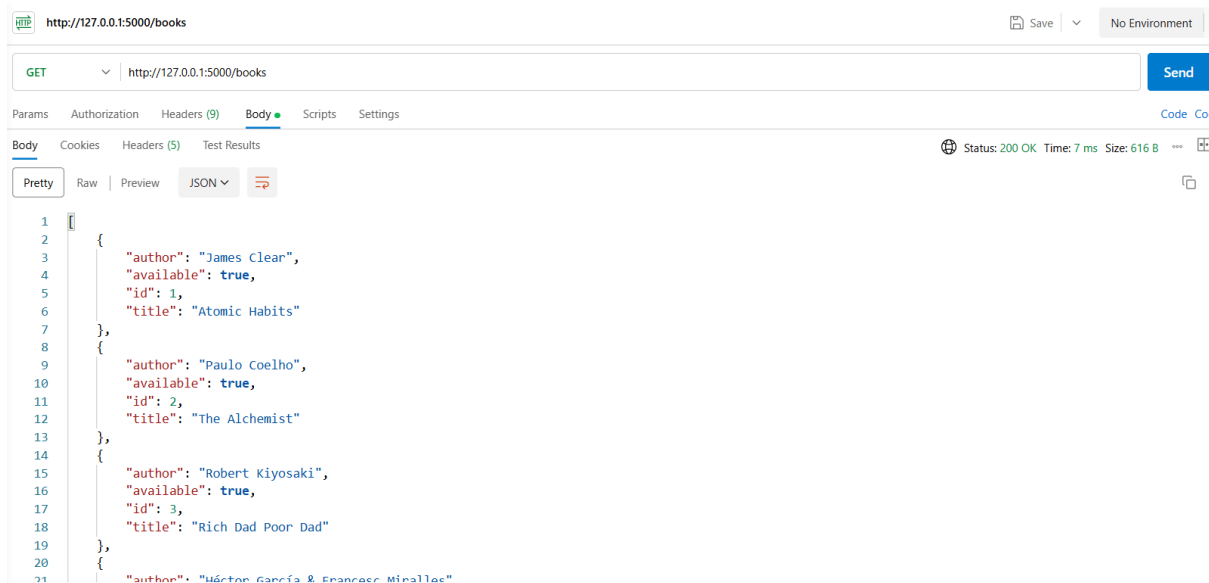
OUTPUT:



POST:



GET



PATCH/PUT

http://127.0.0.1:5000/books/2

Save

No Environmen

PATCH

http://127.0.0.1:5000/books/2

Send

Params

Authorization

Headers (9)

Body

Scripts

Settings

Code

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

1

{

2

-

3

"available": false

4

}

Body

Cookies

Headers (5)

Test Results

Status: 200 OK Time: 7 ms Size: 258 B

Pretty

Raw

Preview

JSON

1

{

2

"author": "Paulo Coelho",

3

"available": false,

4

"id": 2,

5

"title": "The Alchemist"

6

}

7

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

TERMINAL

PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding> & C:/Users/Pavani/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/Pavani/OneDrive/Documents/AI Assisted coding/lab_15_1.py"

127.0.0.1 - - [24/Mar/2026 19:26:37] "GET /books HTTP/1.1" 200 -

127.0.0.1 - - [24/Mar/2026 19:26:43] "GET / HTTP/1.1" 200 -

127.0.0.1 - - [24/Mar/2026 19:28:25] "PUT /books/1 HTTP/1.1" 405 -

127.0.0.1 - - [24/Mar/2026 19:28:30] "GET / HTTP/1.1" 200 -

DELETE

http://127.0.0.1:5000/books/1

Save

No Environment

DELETE

http://127.0.0.1:5000/books/1

Send

Params

Authorization

Headers (9)

Body

Scripts

Settings

Code

Cookies

Body

Cookies

Headers (5)

Test Results

Status: 200 OK Time: 8 ms Size: 198 B

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

TERMINAL

PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding> & C:/Users/Pavani/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/Pavani/OneDrive/Documents/AI Assisted coding/lab_15_1.py"

127.0.0.1 - - [24/Mar/2026 19:28:25] "PUT /books/1 HTTP/1.1" 405 -

127.0.0.1 - - [24/Mar/2026 19:28:30] "GET / HTTP/1.1" 200 -

127.0.0.1 - - [24/Mar/2026 19:29:08] "GET /books HTTP/1.1" 200 -

127.0.0.1 - - [24/Mar/2026 19:29:35] "PUT /books/2 HTTP/1.1" 405 -

127.0.0.1 - - [24/Mar/2026 20:58:20] "PATCH /books/2 HTTP/1.1" 200 -

127.0.0.1 - - [24/Mar/2026 19:29:35] "PUT /books/2 HTTP/1.1" 405 -

127.0.0.1 - - [24/Mar/2026 20:58:20] "PATCH /books/2 HTTP/1.1" 200 -

127.0.0.1 - - [24/Mar/2026 20:59:58] "DELETE /books/1 HTTP/1.1" 200 -

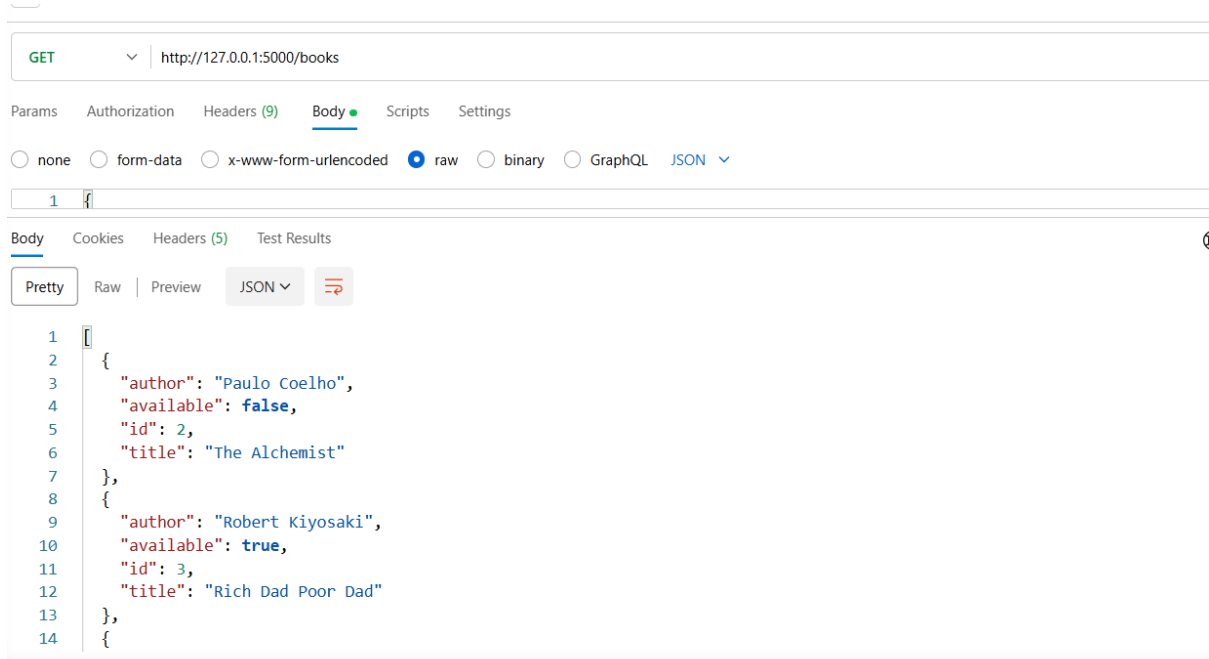
127.0.0.1 - - [24/Mar/2026 21:00:07] "GET / HTTP/1.1" 200 -

127.0.0.1 - - [24/Mar/2026 21:00:08] "GET / HTTP/1.1" 200 -

127.0.0.1 - - [24/Mar/2026 21:00:08] "GET / HTTP/1.1" 200 -

127.0.0.1 - - [24/Mar/2026 21:00:08] "GET / HTTP/1.1" 200 -

127.0.0.1 - - [24/Mar/2026 21:00:09] "GET / HTTP/1.1" 200 -



EXPLANATION:

- Designed a RESTful API using Flask to perform CRUD operations on library books.
- Implemented in-memory data storage using a Python dictionary for efficient data handling.
- Incorporated error handling and validation to ensure reliable and robust API responses.

Task 3 – Employee Payroll API

PROMPT:

develop a python code to create an API for managing employees and their salaries The API should support the following operations: GET /employees → List all employees POST /employees → Add a new employee with salary details PUT /employees/{id}/salary → Update salary of an employee DELETE /employees/{id} → Remove employee from system

CODE:

```
89
90
91 #develop a python code to create an API for managing employees and their salaries
92 # The API should support the following operations:
93 #GET /employees → List all employees
94 #POST /employees → Add a new employee with salary details
95 #PUT /employees/{id}/salary → Update salary of an employee
96 #DELETE /employees/{id} → Remove employee from system
97 from flask import Flask, jsonify, request
98 app = Flask(__name__)
99 employees = {}
100 @app.route('/', methods=['GET'])
101 def home():
102     return jsonify({"message": "Welcome to the Employee Management API!"})
103 @app.route('/employees', methods=['GET'])
104 def get_employees():
105     return jsonify(list(employees.values()))
106 @app.route('/employees', methods=['POST'])
107 def add_employee():
108     data = request.get_json()
109     employee_id = data.get('id')
110     if employee_id in employees:
111         return jsonify({'error': 'Employee already exists'}), 400
112     employees[employee_id] = data
113     return jsonify(data), 201
114 @app.route('/employees/<int:employee_id>/salary', methods=['PUT'])
115 def update_salary(employee_id):
116     if employee_id not in employees:
117         return jsonify({'error': 'Employee not found'}), 404
118     data = request.get_json()
119     employees[employee_id]['salary'] = data.get('salary')
120     return jsonify(employees[employee_id])
121 @app.route('/employees/<int:employee_id>', methods=['DELETE'])
122 def delete_employee(employee_id):
123     if employee_id not in employees:
124         return jsonify({'error': 'Employee not found'}), 404
125     del employees[employee_id]
126
127 employee_id = data.get('id')
128 if employee_id in employees:
129     return jsonify({'error': 'Employee already exists'}), 400
130 employees[employee_id] = data
131 return jsonify(data), 201
132 @app.route('/employees/<int:employee_id>/salary', methods=['PUT'])
133 def update_salary(employee_id):
134     if employee_id not in employees:
135         return jsonify({'error': 'Employee not found'}), 404
136     data = request.get_json()
137     employees[employee_id]['salary'] = data.get('salary')
138     return jsonify(employees[employee_id])
139 @app.route('/employees/<int:employee_id>', methods=['DELETE'])
140 def delete_employee(employee_id):
141     if employee_id not in employees:
142         return jsonify({'error': 'Employee not found'}), 404
143     del employees[employee_id]
144     return jsonify({'message': 'Employee deleted'})
145 if __name__ == '__main__':
146     app.run(debug=True, host='0.0.0.0', port=5000)
```

OBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

TERMINAL

WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

* Running on all addresses (0.0.0.0)

* Running on http://127.0.0.1:5000

* Running on http://10.3.2.185:5000

Press CTRL+C to quit

* Restarting with stat

* Debugger is active!

* Debugger PIN: 705-041-444

127.0.0.1 - - [24/Mar/2026 21:12:34] "GET / HTTP/1.1" 200 -

Press CTRL+C to quit

OUTPUT:

127.0.0.1:5000

Pretty-print ☒

```
{
  "message": "Welcome to the Employee Management API!"
}
```

 http://127.0.0.1:5000/employees

GET

Params Authorization Headers (7) Body Scripts Settings

Query Params

	Key	Value
	Key	Value

Body Cookies Headers (5) Test Results

Pretty Raw Preview JSON

1

 http://127.0.0.1:5000/employees Save

POST

Params Authorization Headers (9) Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "id": 1,
3   "name": "Pavani",
4   "role": "Developer",
5   "salary": 50000
6 }
```

Body Cookies Headers (5) Test Results Status: 201 Created Time: 7 ms

Pretty Raw Preview

```
1 {
2   "id": 1,
3   "name": "Pavani",
4   "role": "Developer",
5   "salary": 50000
6 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

TERMINAL

```
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.3.2.185:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 705-041-444
127.0.0.1 - - [24/Mar/2026 21:12:34] "GET / HTTP/1.1" 200 -
Press CTRL+C to quit
```

GET http://127.0.0.1:5000/employees Save No Environment

Params Authorization Headers (9) Body Scripts Settings Code Cook

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18

```
{
  {
    "id": 1,
    "name": "Pavani",
    "role": "Developer",
    "salary": 50000
  },
  {
    "id": 2,
    "name": "Pallavi",
    "role": "Tester",
    "salary": 45000
  },
  {
    "id": 3,
    "name": "Sanvika",
    "role": "Database operator",
    "salary": 65000
  }
}
```

Body Cookies Headers (5) Test Results Status: 200 OK Time: 6 ms Size: 534 B

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PUT http://127.0.0.1:5000/employees/3/salary

Params Authorization Headers (9) Body Scripts Settings

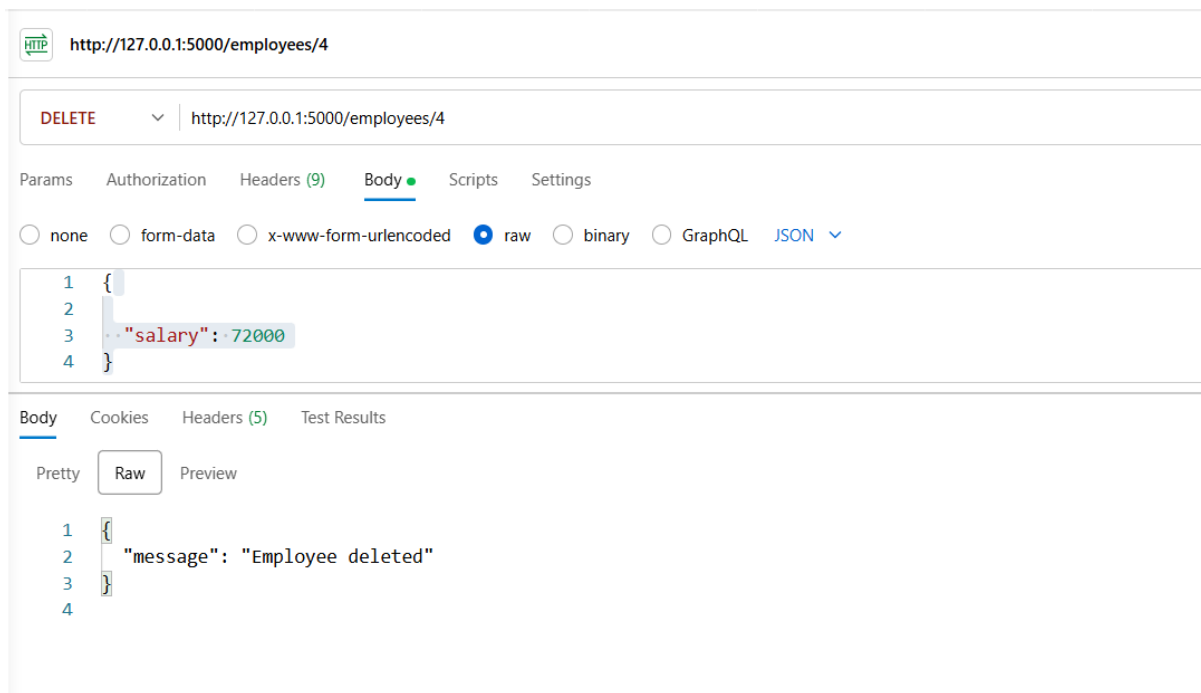
☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "salary": 72000
3 }
4
```

Body Cookies Headers (5) Test Results Status

Pretty Raw Preview

```
1 {
2   "id": 3,
3   "name": "Sanvika",
4   "role": "Database operator",
5   "salary": 72000
6 }
7
```



EXPLANATION:

- Developed a RESTful API using Flask to manage employee records and their salaries.
- Implemented CRUD operations like adding, viewing, updating salary, and deleting employees.
- Used a Python dictionary as in-memory storage to handle employee data.
- Included JSON responses and basic error handling for invalid requests and missing employees.

Task 4 – Real-Time Application: Online Food Ordering API

PROMPT:

develop a python code to design a simpleAPI for an online food ordering system

The API should support the following operations:

GET /menu → List available dishes

POST /order → Place a new order

GET /order/{id} → Track order status

PUT /order/{id} → Update an existing order (e.g., change items)

DELETE /order/{id} → Cancel an order

also implement error handling for invalid requests

CODE:

```
lab_15_1.py > ...
131
132 #develop a python code to design a simpleAPI for an online food ordering system
133 # The API should support the following operations:
134 #GET /menu → List available dishes
135 #POST /order → Place a new order
136 #GET /order/{id} → Track order status
137 #PUT /order/{id} → Update an existing order (e.g.,change items)
138 #DELETE /order/{id} → Cancel an order
139 #also implement error handling for invalid requests
140 from flask import Flask, jsonify, request
141 app = Flask(__name__)
142 menu = {
143     1: {"name": "Pizza", "price": 10.99},
144     2: {"name": "Burger", "price": 7.99},
145     3: {"name": "Pasta", "price": 8.99}
146 }
147 orders = {}
148 @app.route('/', methods=['GET'])
149 def home():
150     return jsonify({"message": "Welcome to the Online Food Ordering API!"})
151 @app.route('/menu', methods=['GET'])
152 def get_menu():
153     return jsonify(menu)
154 @app.route('/order', methods=['POST'])
155 def place_order():
156     data = request.get_json()
157     order_id = len(orders) + 1
158     orders[order_id] = {
159         "items": data.get('items'),
160         "status": "placed"
161     }
162     return jsonify({"order_id": order_id, "status": "placed"}), 201
163
164 @app.route('/order/<int:order_id>', methods=['GET'])
165 def track_order(order_id):
166     if order_id not in orders:
167         return jsonify({'error': 'Order not found'}), 404
168     return jsonify(orders[order_id])
169 @app.route('/order/<int:order_id>', methods=['PUT'])
170 def update_order(order_id):
171     if order_id not in orders:
172         return jsonify({'error': 'Order not found'}), 404
173     data = request.get_json()
174     orders[order_id]['items'] = data.get('items', orders[order_id]['items'])
175     return jsonify(orders[order_id])
176 @app.route('/order/<int:order_id>', methods=['DELETE'])
177 def cancel_order(order_id):
178     if order_id not in orders:
179         return jsonify({'error': 'Order not found'}), 404
180     del orders[order_id]
181     return jsonify({'message': 'Order cancelled'})
182 if __name__ == '__main__':
183     app.run(debug=True,host='0.0.0.0', port=5000)
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
> ▼ TERMINAL
* Serving Flask app 'lab_15_1'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.3.2.185:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 705-041-444
127.0.0.1 - - [24/Mar/2026 21:28:53] "GET / HTTP/1.1" 200 -
```

OUTPUT:

← ↻ ⓘ 127.0.0.1:5000
retty-print ☐

"message": "Welcome to the Online Food Ordering API!"

http://127.0.0.1:5000/menu

GET http://127.0.0.1:5000/menu

Params Authorization Headers (7) Body Scripts Settings

Body Cookies Headers (5) Test Results Status: 200

Pretty Raw Preview JSON

```
1 {
2   "1": {
3     "name": "Pizza",
4     "price": 10.99
5   },
6   "2": {
7     "name": "Burger",
8     "price": 7.99
9   },
10  "3": {
11    "name": "Pasta",
12    "price": 8.99
13  }
14 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

TERMINAL

```
> * Serving Flask app 'lab_15_1'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.3.2.185:5000
Press CTRL+C to quit
* Restarting with stat
```

POST http://127.0.0.1:5000/order

Params Authorization Headers (9) Body ● Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "items": ["Pizza", "Burger"]
3 }
```

Body Cookies Headers (5) Test Results Status: 200

Pretty Raw Preview JSON

```
1 {
2   "order_id": 1,
3   "status": "placed"
4 }
5
```


HTTP  http://127.0.0.1:5000/order/1

PUT  http://127.0.0.1:5000/order/1

Params Authorization Headers (9) **Body** ● Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON 

```
1 {
2   "items": ["Pasta", "Burger"]
3 }
```

Body Cookies Headers (5) Test Results

Pretty Raw Preview JSON  

```
1 {
2   "order_id": 1,
3   "status": "placed"
4 }
5 |
```

HTTP  http://127.0.0.1:5000/order/1

DELETE  http://127.0.0.1:5000/order/1

Params Authorization Headers (7) **Body** Scripts Settings

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body Cookies Headers (5) Test Results

Pretty Raw Preview JSON  

```
1 {
2   "message": "Order cancelled"
3 }
```

EXPLANATION:

- Developed a RESTful API using Flask to manage menu items and customer orders in an online food ordering system.
- Implemented CRUD operations: viewing menu (GET), placing orders (POST), updating orders (PUT), and cancelling orders (DELETE).
- Used in-memory data structures (dictionaries) to store menu and order details.
- Ensured JSON responses and added basic error handling for invalid requests and missing orders.